National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

R10
Searching and Sorting II; Memoization

Problems:

1. Consider the following `make_search` function, which uses the `linear_search` function from Lecture L10A:

   ```
   def make_search(A):
       return lambda x: linear_search(A, x)
   ```

   The following shows a typical use of `make_search`:

   ```
   my_list = [3,41,20,1,5,16,4,0,14,6,17,8,4,0,2]
   my_search = make_search(my_list)
   my_search(14)  # returns True
   ...            # many more calls to my_search
   my_search(30)  # returns False
   ```

   Based on the above use case, complete the function `make_optimized_search` to make an optimized version of `make_search`. Note that the input list `A` must not be modified by your function, or by any subsequent search operations. You should consider only the search and sort algorithms covered in Lecture L10A.

   ```
   def make_optimized_search(A):
       # ???
       return lambda x: ???
   ```

2. Computing Fibonacci numbers.

   (a) Write a function `fib(n)` that returns the nth Fibonacci number, using the following idea: As usual, `fib(0)` should return 0 and `fib(1)` should return 1. In order to

compute `fib(n)`, place the first two Fibonacci numbers in a list, and then use a loop to fill the list with the Fibonacci numbers until the desired `n`.

(b) Observe that we don't really need the whole list to compute the next Fibonacci number. The previous two values suffice. Use this observation to write a version that uses only two variables (do not count the parameter variable `n` and the loop control variable), instead of a list.

3. In Lecture L10A, we saw the memoized n-choose-k function `mchoose`:

```
def make_table(rows, columns):
    table = [None] * rows
    for i in range(rows):
        table[i] = [None] * columns
    return table

def mchoose(n, k):
    mem = make_table(n + 1, k + 1)
    def choose(i, j):
        if j > i:
            return 0
        if mem[i][j] is not None:
            return mem[i][j]
        result = 1 if j == 0 or j == i else (
            choose(i - 1, j) + choose(i - 1, j - 1))
        mem[i][j] = result
        return result
    return choose(n, k)
```

Draw the call tree for the evaluation of the function call `mchoose(7, 4)`. Show only the calls to the inner function `choose`, and for calls whose return values can be found in the table `mem`, indicate so.

What is the order of growth in time of `mchoose(n, k)`?

What is the order of growth in space of `mchoose(n, k)`?